Week 4

W4. 구현 자동화
설계부터 코드, 그리고 배포까지

로디 박한근

About

박한근

- 퍼스트블럭컨설팅 대표 컨설턴트
- 로디 서비스 & 디자인 아카데미 강사
- 전 바이널 서비스 디자인 수석 컨설턴트
- 전 이모션 전략기획팀 수석 컨설턴트
- 웹기획자 / 서비스 디자이너 / 프로젝트 매니저 / 엔지니어로 14년째 활동 중
- SKT, 삼성전자, 현대자동차, 삼성카드, 제일모직 등 대기업 프로젝트 제안/구축 PM
- 행정안전부, 한국도로공사, 주택금융공사 등 정부/공공기관 프로젝트 제안/구축 PM
- 서울대 / 연세대 / 국민대 디자인 대학원 특강
- Linkedin : www.linkedin.com/in/rodipark

이번 주차 개요

설계 → App Router 코드 → Vercel 배포

이번 주차 한 줄 요약

테마
구현 자동화 — 설계, 코드, 배포의 자동화 체이닝 (App Router 단일 구성)

총 시간 / 비율
5시간(300분), 강의:실습 = 15:85

구현 아키텍처
Next.js 14 App Router 풀스택 단일 구성 — 별도 Express 백엔드 없음

기반 프레임워크
AP-Framework V0.42 (Group B 병렬 + 배포 트러블슈팅 통합)

학습 목표

- 구현 설계 문서 3건(인프라·DB·디자인)을 Group B로 동시 산출한다
- Claude Design 프로토타입 → Claude Code 핸드오프를 수행한다
- 프론트·백엔드를 바이브 코딩(Sonnet) + 목업 데이터로 완성한다
- npm run dev로 목업 전체 흐름을 로컬에서 확인한다
- Supabase 연동(목업→실DB 교체) 후 Vercel에 배포한다

Phase 3 Gate-Check — W3 산출물 재사용

- #3 서비스기획서 / #6 기능명세서 / #7 API스펙 / #9 화면설계서 — 완료(W3)
- **#11 시스템정의서 — W3 Group A에서 이미 생성, W4는 입력으로만 재사용**
- #7·#9는 실습 1(Group B 병렬)의 절대 전제

핵심 이번 주차는 '코드가 먼저, 연결은 나중' — 목업으로 개발 후 Supabase·Vercel 연결.

이론
꼭 알아야 할 개념

분리형 vs App Router 단일 구성

(구) 분리형 구성

- src/frontend + src/backend 분리
- 별도 Express 서버 (server.js)
- pages/api 프록시 + Root Directory
- pg 직접 연결 → IPv6 에러 → REST 어댑터
- 트러블슈팅 4건이 항상 따라옴

(현) App Router 단일

- 단일 Next.js 앱 하나
- 별도 서버 없음 — app/api Route Handler
- 단일 프로젝트, 프록시 불필요
- Supabase JS SDK(HTTPS) — IPv6 문제 원천 없음
- 트러블슈팅 2건으로 감소

개발 프레임워크: Next.js란?

개념	쉽게 말하면	본 과정에서
React	화면(컴포넌트)을 그리는 부품 라이브러리	components/ 의 버튼·카드
Next.js	React에 라우팅+서버+빌드를 더한 프레임워크	앱 전체의 뼈대
App Router	app/ 폴더 구조가 곧 URL이 되는 최신 방식	app/page.tsx=홈
서버/클라이언트 컴포넌트	서버 렌더 기본 / 'use client' 상호작용	목록 / 장바구니 담기
Route Handler	app/api/.../route.ts = 별도 서버 없는 API	app/api/products

Next.js · App Router 핵심

- **React=부품, Next.js=그 부품으로 짓는 집, App Router=폴더가 곧 주소**
- 폴더 구조가 곧 URL — app/products/[id]/page.tsx → /products/123
- 화면(page.tsx)과 API(route.ts)가 같은 app/ 안에 산다
- **→ 별도 백엔드 서버 없이 한 앱에서 프론트+API 처리 (단일 구성)**

핵심 TypeScript=타입 안전한 JS, Tailwind=유틸리티 CSS (한 줄씩만 짚고 넘어감).

소스 디렉토리: src/ 구조 (AP-Rodi-Shop)

```
src/
├── frontend/              <- Next.js 14 (App Router) — 화면
│   ├── app/               페이지(page.tsx) = URL 구조
│   ├── components/        재사용 UI (ui/ layout/ product/ ...)
│   ├── lib/               api.ts · auth.tsx · utils.ts
│   └── pages/api/[...path].ts   백엔드 프록시(분리형 전용)
└── backend/               <- Express.js (분리형일 때만)
    ├── server.js  config/  routes/  controllers/
    └── models/  middleware/  services/  sql/
```

routes=창구, controllers=주방장, models=창고, middleware=검표.

/backend 있을 때 vs 없을 때

/backend 있음 (분리형)

- 백엔드: src/backend Express(server.js)
- API 구성: routes→controllers→models 3단
- 연결: pages/api 프록시가 Express로 위임
- DB: config/db.js REST 어댑터(run_query)
- 실행: 프론트+백엔드 2개 의식

/backend 없음 (App Router 단일)

- 백엔드: 없음 — app/api/route.ts
- API 구성: route.ts + lib/repositories
- 연결: 같은 앱이라 상대경로 /api/ 직접
- DB: lib/supabase supabase-js SDK
- 실행: 단일 Next 앱 하나

같은 '상품 목록' 엔드포인트 두 방식

```
[분리형]
 fetch('/api/products') -> pages/api 프록시
   -> Express routes/products -> controllers
   -> models (run_query SQL) -> 응답

[App Router 단일]
 fetch('/api/products') -> app/api/products/route.ts
   -> lib/repositories/productRepo
   -> lib/supabase (supabaseAdmin.select()) -> 응답
```

예제(AP-Rodi-Shop)=분리형, 실습 산출물=단일. 폴더가 다른 이유.

바이브 코딩 (Vibe Coding · Claude Sonnet)

무엇인가
설계 문서를 컨텍스트로, AI와 대화하듯 반복(생성→확인→피드백→수정)하며 코드 완성

방법
완벽한 프롬프트 1번이 아니라, 결과를 보며 '이 부분 이렇게 바꿔줘'를 빠르게 왕복

모델 분담
설계·아키텍처(실습 1·2)는 상위 모델 / 반복 코드 생성(실습 3·4)은 Sonnet

왜 Sonnet
프론트·백엔드 반복 생성은 속도·비용 효율이 중요 → Sonnet이 적합

왜 목업 데이터로 먼저 개발하는가

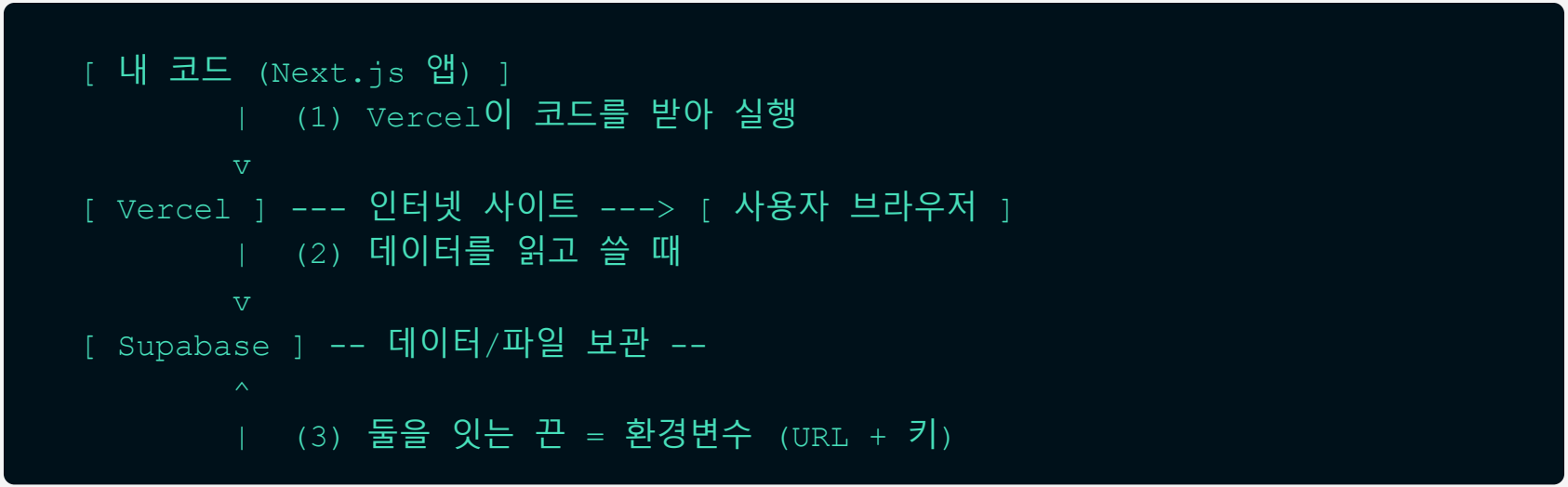
문제
외부 서비스(Supabase) 연결은 수강생이 가장 자주 막히는 지점

해법
프론트(lib/mock)·백엔드(목업 repository)로 코드를 먼저 완성

교체
실습 6에서 repository 한 겹만 Supabase SDK로 교체 → 화면·라우트 그대로

효과
문제를 '코드'와 '연결'로 분리 → 디버깅이 쉬워짐

내 코드 · Vercel · Supabase 한 장 그림



막히는 분의 90%는 코드가 아니라 '이 끈'을 잘못 묶은 것이다.

Supabase — 빌려 쓰는 클라우드 DB

무엇인가
클라우드에 이미 켜져 있는 PostgreSQL을 빌려 쓰는 서비스 (DB+인증+Storage)

무엇을 받나
Project URL 1개 + API 키 2종류(anon / service_role)

키가 왜 2개
anon=공개용(브라우저 OK), service_role=만능 마스터키(서버 전용, 노출 금지)

비유
프로젝트=은행 금고, URL=지점 주소, service_role=지점장 마스터키

Vercel — push하면 사이트가 뜨는 자판기

무엇인가
Next.js 앱을 인터넷 주소로 띄워주는 호스팅 (서버리스)

어떻게 배포
GitHub 저장소 연결 → git push 할 때마다 자동 빌드·배포

환경변수는
.env.local은 Vercel이 모른다 → 대시보드에도 똑같이 등록해야 연결됨

백엔드는
app/api Route Handler가 Vercel Functions(서버리스 함수)로 실행

Supabase 요금제 (2026-06 기준)

플랜	월 요금	핵심 포함량 / 비고
Free	$0	DB 500MB·MAU 5만·파일 1GB·프로젝트 2개 / 1주 미사용 자동중지
Pro	$25~	DB 8GB·MAU 10만·파일 100GB·무제한 프로젝트 / 자동중지 없음
Team	$599~	Pro + SOC2/ISO·14일 백업·우선지원
Enterprise	협의	맞춤 한도·SLA·HIPAA·전용지원

Vercel 요금제 (2026-06 기준)

플랜	월 요금	핵심 포함량 / 비고
Hobby	$0	전송 100GB·함수 1M·Edge 1M / 개인·비상업용 전용
Pro	$20 / 사용자	전송 1TB·Edge 10M·$20 크레딧 / 상업용 가능, 초과 $0.15/GB
Enterprise	협의	SLA·보안·규정준수·전용지원

수업에서 꼭 짚을 2가지 + 권장 조합

- **Supabase Free: 1주 미사용 시 자동 일시중지 → /api/health 실패 가능(Restore로 복구)**
- **Vercel Hobby: 개인·비상업용 전용 → 상품 파는 커머스는 Vercel Pro 필요**
- 학습·실습 = Supabase Free + Vercel Hobby ($0) — W4 실습 전체 무료 완주 가능
- **상업 운영 = Supabase Pro($25) + Vercel Pro($20) ≈ 월 $45부터**

핵심 정확한 최신가는 supabase.com/pricing · vercel.com/pricing 확인.

실습

설계 → 코드 → 배포, 순서대로

이번 주차 진행 순서 (7단계)

(1) 구현 설계 산출	P-409 Group B -> 인프라+DB+디자인
(2) 프로토타입	Claude Design -> Claude Code 핸드오프
(3) 프론트 개발	바이브 코딩(Sonnet) · 목업 데이터
(4) 백엔드 개발	바이브 코딩(Sonnet) · 목업 repository
(5) 로컬 확인	npm run dev (걸어다니는 뼈대)
(6) Supabase 연동	목업 -> 실DB 교체 (repository 한 겹)
(7) Vercel 배포	GitHub 연동 + 환경변수 -> 라이브 URL

코드를 목업으로 먼저 완성하고, Supabase·Vercel은 마지막에 연결한다.

실습 1 · 구현 설계 산출 (30분)

P-409 Group B — 인프라 + DB + 디자인 동시 생성

P-409 Group B — 왜 병렬이 가능한가

- **#10 인프라 · #12 DB설계 · #13 디자인스타일가이드를 동시 생성**
- 세 산출물은 서로를 입력으로 받지 않음 → 독립적
- 모두 #3/#6/#7/#9 + #11(시스템정의서)을 공통 입력으로 사용
- Task 3개를 동시에 띄워 병렬 실행 → 이후 코드·연동의 공통 입력

핵심 순차 ~45분 → 병렬 ~15분 (약 3배 가속, 토큰은 약 1.5배).

Group B 정합성 검증

검증 항목	확인 방법
#10 ↔ #12	인프라의 Supabase ENV가 DB 설계(PostgreSQL 16)와 정합한가
#10 ↔ #13	정적 자산/폰트 경로가 디자인 가이드와 일치하는가
#12 ↔ #13	DB enum 값이 디자인 가이드 배지 컬러와 매핑되는가
아키텍처 일관성	3건 모두 App Router 단일 구성을 전제하는가

실습 1 핵심 프롬프트

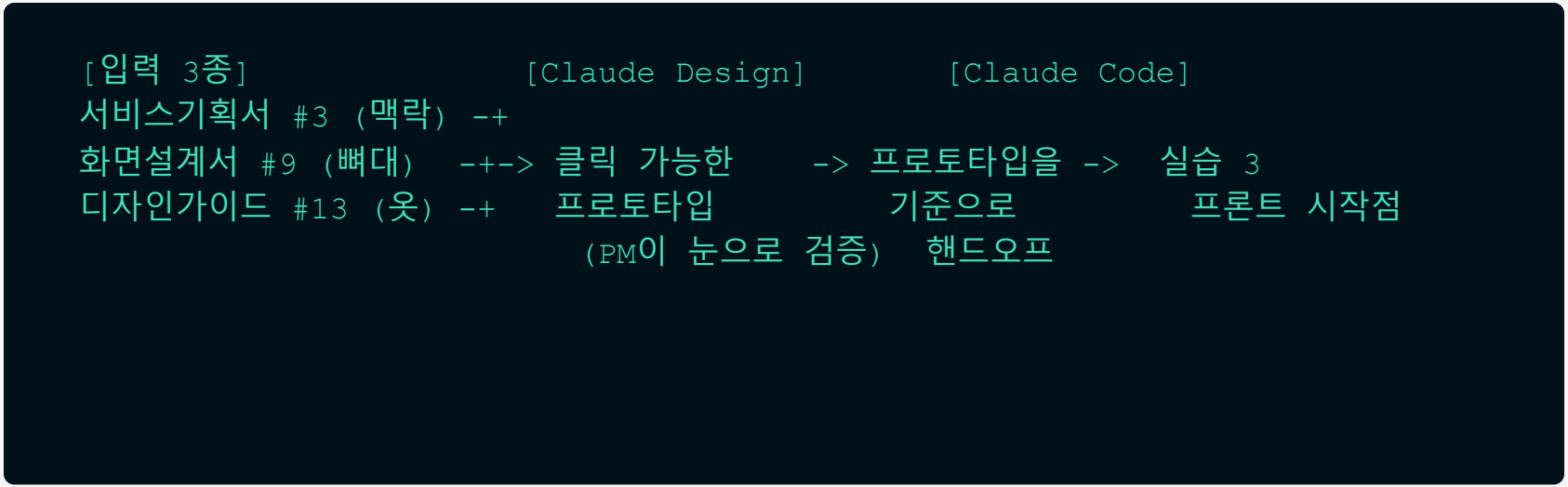
프롬프트 P-409 · Group B 병렬

```
claude "P-409 Group B 병렬 실행을 시작해줘.
아키텍처 전제: Next.js App Router 풀스택 단일 구성
(별도 Express 없음, API는 app/api/ Route Handler, DB는 Supabase JS SDK).
시스템정의서(#11)를 공통 통합 기준점으로 참조해줘.

다음 3개를 Task 도구로 병렬 생성:
1. 인프라아키텍처(#10): 구성도 + 환경변수 목록(노출 경계 명시)
   + CI/CD → 03.구현문서/인프라아키텍처.md
2. 데이터베이스설계서(#12): ERD(Mermaid) + 7테이블 DDL
   + SQL Editor용 schema.sql + 시드 → .../데이터베이스설계서.md
3. 디자인스타일가이드(#13): 컬러/타이포/컴포넌트
   + tailwind.config.ts 전체 → .../디자인스타일가이드.md

3개 Task로 병렬 실행하고, 완료 후 .progress.md를 일괄 업데이트해줘."
```

실습 2 · 프로토타입 핸드오프 (30분)

Claude Design → Claude Code 핸드오프

설계-프로토타입-코드 핸드오프



화면설계서=뼈대, 디자인가이드=옷, 서비스기획서=맥락. 셋을 합쳐 프로토타입.

실습 2 핵심 프롬프트 (1) Claude Design

프롬프트 프로토타입 생성

```
[Claude Design 프롬프트]
다음 세 문서를 기준으로 클릭 가능한 프로토타입을 만들어줘.
- 서비스기획서(#3): 맥락·타겟·핵심 가치 → 무엇을 강조할지
- 화면설계서(#9): SCR-001~012 와이어프레임/상태 → 화면 구조 기준
- 디자인스타일가이드(#13): 컬러/타이포/컴포넌트 → 시각 기준

요구:
1. SCR-ID를 그대로 화면 단위로, 화면 간 이동(링크) 연결해 클릭 흐름 구성
2. 디자인가이드의 컬러/폰트/버튼·뱃지·카드 스타일 엄격 적용
3. 핵심 흐름(카탈로그→상세→장바구니→결제) 우선 완성
4. 로딩/빈/에러 상태도 화면설계서대로 표현
```

실습 2 핵심 프롬프트 (2) 핸드오프

프롬프트 Claude Code

```
claude "Claude Design 프로토타입을 핸드오프 입력으로 받아,
프론트엔드 개발 기준을 정리해줘.

핸드오프 원칙:
- 프로토타입을 '시각적 명세'로 삼되, 최종 코드는
  화면설계서(#9) SCR-ID, 디자인가이드(#13) 토큰 기준으로 정합화.
- 인라인 스타일을 베끼지 말고 tailwind 토큰 +
  components/ui/ 컴포넌트로 치환할 계획을 세워줘.
- 구조: Next.js 14 App Router. 서버 컴포넌트 기본,
  상호작용 컴포넌트만 'use client'.

프로토타입(시각)↔화면설계서(구조)↔디자인가이드(토큰)
3중 정합 기준을 한 페이지로 요약해줘."
```

3단계 + 핸드오프 핵심 원칙

Step 1 — 등록·생성
입력 3종을 Claude Design에 넣어 SCR-ID별 클릭 프로토타입 생성

Step 2 — PM 검수
흐름·SCR 매핑·스타일·상태 표현을 코드 전에 점검

Step 3 — 핸드오프
프로토타입을 '명세'로 참조 → 디자인 토큰·공통 컴포넌트로 정규화

핵심
복붙이 아니라 명세 참조. Claude Design=시각 검증 / Claude Code=구현

실습 3 · 프론트엔드 개발 (40분 · Sonnet)

바이브 코딩 · 목업 데이터로 화면 완성

목업 우선 — 데이터 없이 화면부터

- **프론트는 '데이터가 어떻게 생겼는지'만 알면 화면을 만든다**
- lib/mock/ 의 더미 데이터(상품·주문)로 12개 화면 완성
- lib/data/index.ts 로 데이터 접근을 추상화 (시그니처 고정)
- **실습 6에서 데이터 출처를 목업 → /api(실제)로 한 줄만 교체**

핵심 바이브 코딩 팁: 화면 단위로 '홈부터 보여줘 → 확인 → 수정' 반복.

실습 3 핵심 프롬프트 (1) 기반

프롬프트 P-401 · 프론트 기반

```
claude "P-401 프론트엔드 코드 생성기 (App Router, 바이브 코딩).
핸드오프 프로토타입 + 시스템정의서(#11)·디자인가이드(#13) 기반
Next.js 14 App Router + TS + Tailwind 코드 생성.
이 단계는 Supabase 없이 '목업 데이터'로 화면을 완성한다.

원칙:
- 서버 컴포넌트 기본, 상호작용만 'use client'
- 클라이언트 혹은 <Suspense> 래핑
- next.config.js: images.unoptimized = true
- output:'export' 금지

생성할 파일:
1. package.json / tsconfig / next.config / tailwind.config.ts(#13)
2. app/globals.css, app/layout.tsx, types/index.ts
3. lib/mock/products.ts, lib/mock/orders.ts (더미 데이터)
4. lib/data/index.ts -- 데이터 접근 추상화(지금은 목업 반환,
   실습 6에서 /api 호출로 교체 가능하게 시그니처 고정)
```

실습 3 핵심 프롬프트 (2) 페이지

프롬프트 P-401 · 12화면

```
claude "핸드오프 프로토타입 + 화면설계서(#9)의 12개 화면을
App Router 페이지로 구현. 데이터는 lib/data/(현재 목업)에서.

생성할 파일:
- components/layout/ product/ cart/ auth/ admin/
- app/page.tsx (SCR-001 홈/카탈로그) -- 목업 상품 목록
- app/products/[id]/page.tsx (SCR-002, 동적 라우트)
- app/cart, app/checkout, app/orders/[id]/complete
- app/auth/login, app/auth/register, app/my-purchases
- app/admin/ 하위 (app/admin/layout.tsx 포함)

SCR-001~012 모두 매핑. 디자인가이드 스펙 엄격 준수."
```

실습 3 품질 체크포인트

- 12개 화면(SCR-001~012)이 목업 데이터로 모두 렌더됨
- 데이터 접근이 lib/data/로 추상화 (실습 6 교체 가능)
- 프로토타입 인라인 스타일이 디자인 토큰/공통 컴포넌트로 정규화
- 클라이언트 훅 사용처가 'use client' + Suspense 처리
- output:'export' 미사용 (동적 라우트 호환)

실습 4 · 백엔드 개발 (40분 · Sonnet)

바이브 코딩 · 목업 repository로 API 완성

백엔드도 목업 우선

- **Route Handler는 API스펙(계약)만 지키면 된다**
- lib/repositories/ 를 목업 구현(메모리/JSON)으로 먼저 작성
- 함수 시그니처를 고정 → 실습 6에서 내부만 Supabase SDK로 교체
- **Route Handler·프론트는 손대지 않는다**
- 인증은 Supabase Auth(@supabase/ssr 쿠키 세션)로 통일 — 자체 JWT/bcrypt 없음, GET은 dynamic='force-dynamic'

핵심 repository 한 겹만 바꾸면 라우트는 그대로 동작한다.

실습 4 핵심 프롬프트 (1) 기반

프롬프트 P-402 · 백엔드 기반

```
claude "P-402 백엔드 코드 생성기 (App Router, 바이브 코딩).
시스템정의서(#11)·데이터베이스설계서(#12) 기반 백엔드 기반 코드.
이 단계는 Supabase 없이 '목업 repository'로 API를 동작시킨다.

원칙:
- 모든 API는 app/api/.../route.ts (별도 Express 없음)
- 데이터 접근은 lib/repositories/ 만 통해서. 지금은 목업 구현이되
  함수 시그니처를 고정 → 실습 6에서 Supabase SDK로 교체 가능하게.
- 인증: Supabase Auth(@supabase/ssr 쿠키 세션)로 통일. 자체 bcrypt·JWT(JWT_SECRET) 금지.
- GET Route Handler: export const dynamic = 'force-dynamic'

생성할 파일:
1. lib/api/response.ts, withAuth.ts(supabase.auth.getUser()), withAdmin.ts(profiles.role)
2. middleware.ts (@supabase/ssr 세션 쿠키 갱신) -- 실습 6에서 실제 연결, 지금은 골격
3. lib/repositories/ (profile/product/cart/order/download, 목업)
4. supabase/schema.sql (auth.users + public.profiles 트리거, 실습 6에서 사용)"
```

실습 4 핵심 프롬프트 (2) 라우트

프롬프트 P-402 · Route Handler

```
claude "API스펙(#7)의 엔드포인트를 app/api/ Route Handler로 구현.
데이터는 lib/repositories/(현재 목업) 통해 접근.

생성할 라우트(예시):
- 자체 register|login|refresh 라우트는 만들지 않는다 (회원가입/로그인은
  클라이언트에서 supabase.auth.signUp / signInWithPassword 호출)
- app/auth/callback/route.ts (소셜 로그인 OAuth code → 세션 교환)
- app/api/products/route.ts (목록+검색+페이지네이션), products/[id]
- app/api/cart/route.ts (GET/POST/DELETE)
- app/api/orders/route.ts (POST=pending), orders/[id]
- app/api/payments/confirm/route.ts (실습 6에서 토스 실연동)
- app/api/admin/products/route.ts 등
- app/api/downloads/[orderId]/[productId]/sign/route.ts

핵심:
- 보호 라우트는 withAuth 래퍼로 감싸기
- 주문 2단계 (pending → confirm → paid)
```

상품 목록 Route Handler (목업 repository)

```
// app/api/products/route.ts
import { productRepo } from '@/lib/repositories/productRepo'
export const dynamic = 'force-dynamic'

export async function GET(req) {
  const { data, total } = await productRepo.list({ q, page, limit:20 })
  return NextResponse.json({ status:'ok', data, total, page })
}
```

실습 6에서 productRepo 내부만 Supabase로 교체 → 이 라우트는 그대로.

실습 5 · 로컬 확인 (15분)

npm run dev — 목업으로 전체 흐름 점검

로컬 확인 — 걸어다니는 뼈대

실행
npm install → npm run dev → http://localhost:3000

점검
홈 → 상세 → 장바구니 → 결제 흐름이 목업으로 끊김 없이 도는가

API
curl /api/products → 목업 JSON 반환 확인

의미
여기서 되면, 이후 막힘은 '코드'가 아니라 '연결' 문제로 분리됨

실습 5 핵심 프롬프트 (이슈 수정)

프롬프트 목업 디버깅

```
claude "npm run dev 로 확인 중 [화면/에러]가 발생했어.
콘솔 로그는 [붙여넣기].
목업 데이터 기준으로 이 문제만 고쳐줘 (아직 Supabase 연결 전이야)."
```

실습 6 · Supabase 연동 (35분)

목업 → 실DB 교체 (repository 한 겹)

Supabase 연동 5단계

1 프로젝트·키
supabase.com New project → URL·anon·service_role 확보 (값은 .AP-key.md 에 보관)

2 환경변수
.AP-key.md → .env.local 복사 (service_role엔 NEXT_PUBLIC_ 금지)

3 스키마
schema.sql을 SQL Editor에 Run → 테이블 생성

4 교체
repository 목업 → Supabase SDK (시그니처 유지)

키 보관은 .AP-key.md 하나면 충분

- **Supabase URL·anon·service_role 값은 .AP-key.md(비공개)에 적어 둔다**
- 교안·문서에는 실제 값을 기재하지 않는다
- .env.local 은 .AP-key.md 에서 복사해 채운다 (대시보드 재방문 불필요)
- **.gitignore 에 .env.local 과 .AP-key.md 둘 다 포함 (Git에 안 올림)**
- supabase-js(REST) 사용 → Vercel IPv4에서도 IPv6 이슈 없음

핵심 service_role·TOSS_SECRET 에는 절대 NEXT_PUBLIC_ 금지. 인증은 Supabase Auth → JWT_SECRET 불필요.

실습 6 핵심 프롬프트 (1) 환경변수

프롬프트 .AP-key.md → .env.local

```
claude ".AP-key.md 에 적어 둔 Supabase 값을 읽어서
.env.local 과 .env.example 을 만들어줘 (앱이 src/frontend/ 면 그 안에).

# Supabase (.AP-key.md 에서 복사)
NEXT_PUBLIC_SUPABASE_URL=          # Project URL
NEXT_PUBLIC_SUPABASE_ANON_KEY=     # anon (publishable, 브라우저용)
SUPABASE_SERVICE_ROLE_KEY=         # service_role (서버 전용, NEXT_PUBLIC_ 금지)
DATABASE_URL=                      # (선택) 마이그레이션/직접 연결용
# 사이트/결제
NEXT_PUBLIC_SITE_URL=http://localhost:3000   # 배포 후 vercel.app 로 갱신
NEXT_PUBLIC_TOSS_CLIENT_KEY=       # (결제 연동 시)
TOSS_SECRET_KEY=                   # (결제 연동 시)

# 인증은 Supabase Auth 사용 → 별도 JWT_SECRET 만들지 마.
.env.local 은 .AP-key.md 실제 값으로 채우고, .env.example 은 키 이름만.
.gitignore 에 .env.local 과 .AP-key.md 포함 확인."
```

실습 6 핵심 프롬프트 (2) repository 교체

프롬프트 목업 → 실DB

```
claude "Supabase JS SDK(@supabase/supabase-js) 설치하고,
lib/supabase/server.ts (service_role, supabaseAdmin),
lib/supabase/client.ts (anon, 브라우저용)를 만들어줘.

그리고 lib/repositories/ 의 목업 구현을 Supabase SDK로 교체:
- productRepo.list/get → supabaseAdmin.from('products').select().ilike().range()
- user/cart/order/downloadRepo 도 동일 인터페이스 유지하며 교체
- 함수 시그니처(입출력)는 바꾸지 마
  (Route Handler/프론트가 수정 없이 동작해야 함)
- lib/data/index.ts 도 목업 → /api 호출로 전환

연결 확인용 app/api/health/route.ts 도 만들어줘
({ok:true, supabase:{reachable:true, schemaReady:true}} 반환, force-dynamic)."
```

핵심: repository 한 겹만 교체

- lib/supabase/server.ts(service_role) / client.ts(anon) 생성
- productRepo.list → supabaseAdmin.from('products').select().range()
- **함수 입출력은 그대로 → 화면·Route Handler 수정 없음**
- /api/health → {ok:true, supabase:{reachable:true, schemaReady:true}}
- [심화] 토스 결제 confirm · Storage Signed URL(TTL 300s)

핵심 추상화 덕분에 '한 겹 교체'로 앱이 실데이터로 살아난다.

실습 7 · Vercel 배포 (25분)

STEP 1 Root Directory → STEP 2 환경변수 → STEP 3 배포·검증

배포 실패의 2대 원인 (먼저 칠판에)

- **① Root Directory 미지정 — 앱이 src/frontend/ 서브디렉토리에 있음**
- → 저장소 최상단에서 빌드 시도 → package.json 못 찾음 → 실패
- **② Vercel은 내 .env.local 을 모름 — 환경변수 미등록**
- → 대시보드에 등록 + Redeploy 해야 연결됨
- **이 둘만 잡으면 배포는 끝난다**

핵심 vercel.com/new → 저장소 Import → 빌드 설정에서 Root Directory 지정.

STEP 1 빌드 설정 — Root Directory (가장 중요)

설정	값	비고
Root Directory	src/frontend	Edit 눌러 직접 지정 (모노레포 핵심)
Framework Preset	Next.js	자동 감지
Build Command	next build (기본값)	그대로
Output Directory	.next (기본값)	그대로
Install Command	npm install (기본값)	그대로

STEP 2 환경변수 등록 (.AP-key.md 출처)

Name	값 출처	노출
NEXT_PUBLIC_SUPABASE_URL	.AP-key.md (Project URL)	공개
NEXT_PUBLIC_SUPABASE_ANON_KEY	.AP-key.md (anon)	공개
SUPABASE_SERVICE_ROLE_KEY	.AP-key.md (service_role)	비공개
NEXT_PUBLIC_SITE_URL	배포 후 vercel.app 로 갱신	공개
NEXT_PUBLIC_TOSS_CLIENT_KEY / TOSS_SECRET_KEY	토스(결제 단계에만)	공개/비공개

STEP 3 배포 & 검증 (라이브에서 직접)

확인 항목	URL	기대 결과
홈	/	홈 화면 · 브랜드 컬러/폰트 정상
목록	/books	Supabase 데이터 목록 표시
상세	/books/b1	상세 렌더, 없는 id는 404
Supabase 연결	/api/health	{ok:true, supabase:{reachable:true,...}}
결제 동선	/cart→/checkout→success	클릭 이동 동작

배포 트러블슈팅

이 프로젝트 단골 원인

- 빌드 실패(파일 없음) → Root Directory=src/frontend
- /books 비어 보임 → ENV 미등록 → Redeploy
- /api/health reachable:false → service_role 키 확인
- 한글 폰트 미적용 → Pretendard CDN/로컬 호스팅

App Router라 원천 소멸/유지

- Body Escaping — 거의 소멸 (req.json)
- Supabase IPv6 — 원천 소멸 (supabase-js REST)
- bcrypt 네이티브 — 소멸 (Supabase Auth가 처리)
- ENAMETOOLONG — 유지 → .vercelignore 한글 제외

과제 & 정리

과제 목록 (W5 전 제출)

#	과제	제출
1	P-409 Group B 병렬 (#10/#12/#13)	단일 커밋
2	Claude Design 프로토타입 + 핸드오프	링크/스샷
3	프론트 바이브 코딩 (Sonnet, 목업)	GitHub
4	백엔드 바이브 코딩 (Sonnet, 목업)	GitHub
5	로컬 확인 (npm run dev)	녹화/스샷
6	Supabase 연동 (목업→실DB)	스크린샷
7	Vercel 배포 + 트러블슈팅	라이브 URL

4주차 핵심 메시지

코드가 먼저, 연결은 나중
목업으로 프론트·백엔드 완성 → Supabase·Vercel은 마지막에 연결

repository 한 겹 교체
추상화 덕분에 목업→실DB 교체 시 화면·라우트는 그대로

막힘의 90%는 '연결'
로컬은 되는데 배포 안 됨 = Vercel ENV 미등록 + Redeploy

역할 분담
설계는 상위 모델 / 반복 코딩은 Sonnet(바이브 코딩)

Thank you

로디 서비스 & 디자인 아카데미